Parallel Gradient Computation and Synchronization: Enhancing the Efficiency of Distributed Training for LLMs

Hao Li, Hao Jiang, *Member, IEEE*, Jing Wu, *Member, IEEE*, Guiao Yang, and Jian Zhang, *Member, IEEE*

***Abstract*—As the size of large language models (LLMs) increases, the limitations of a single data center, such as constrained computational resources and storage capacity, have made distributed training across multiple data centers the preferred solution. However, a primary challenge in this context is reducing the impact of gradient synchronization on the training efficiency across multiple data centers. In this work, we propose a distributed training scheme for LLMs, named parallel gradient computation and synchronization (PGCS). Specifically, while one expert model is being trained to compute gradients, another expert model performs gradient synchronization in parallel. In addition, a gradient synchronization algorithm named BLP is developed to find the optimal gradient synchronization strategy under arbitrary network connectivity and limited bandwidth across multiple data centers. Ultimately, the effectiveness of PGCS and BLP in enhancing the efficiency of distributed training is demonstrated through comprehensive simulations and physical experiments.**

***Index Terms*—Distributed training, gradient synchronization, large language models, expert models.**

I. Introduction

IN RECENT years, large language models (LLMs) have emerged as revolutionary tools that excel in a range of tasks such as translation, summarization, and question answering [1]. A clear trend in their development is the increasing size of these models [2]. This trend is driven by several factors, including enhanced generalization across tasks, improved output accuracy, and the ability to process multi-modal data [3], [4], [5]. However, as the size of LLMs continues to grow to trillions of parameters [6], the associated challenges in terms of computational and storage demands, energy consumption, and training time become increasingly significant [7]. Consequently, the efficient training of larger LLMs has become a crucial topic.

There are two main infrastructures for training LLMs: centralized training within a single data center and distributed training across multiple data centers [8]. Centralized training in a single data center involves performing all processes related to LLM training, including the storage and management of all data such as parameters, gradients, optimizer states, and training data, within one centralized location [9]. This architecture simplifies resource management and reduces training complexity [10]. However, it has several disadvantages. First, the scale of accelerators in a single data center is limited by resources such as power supply, physical space, and cooling capacity [11]. As the size of LLMs increases, the computational and storage resources in a single data center become inadequate [12]. Additionally, leading companies such as Alibaba [13], Microsoft [14], and Google [15] store their training data in multiple geographically dispersed data centers [16]. The size of training data in each data center is substantial, and sensitive data, such as medical and financial information, is subject to privacy restrictions, making it challenging to consolidate training data from different locations or organizations into a single data center [8], [17]. Consequently, the challenges of resource constraints and data consolidation make centralized training of LLMs in a single data center unsuitable [17].

In contrast, distributed training across multiple data centers presents a promising alternative. This architecture integrates geographically dispersed data centers into a uniform training framework [18]. It not only overcomes the limitations of a single data center in terms of computing power, storage capacity, and energy consumption [7], but also enables LLM training using local data, which is particularly suitable for handling sensitive data [19]. Furthermore, training LLMs requires substantial financial investments, including GPU purchase, construction of high-speed network connections, and coverage of system operations costs, which can be prohibitive for many small and medium companies [6]. Using this way, these companies can form alliances to share GPUs across their respective data centers, thus reducing barriers to entry in the field of LLM training and fostering open innovation [6]. Therefore, the above advantages make distributed training for LLMs across multiple data centers a more practical scheme.

Efficient distributed training for LLMs across multiple data centers poses significant challenges:

Synchronization overhead: Frequent gradient synchronization within or across data centers introduces substantial latency, which can hinder training efficiency as model size and accelerator scale increase.

Received 19 November 2024; revised 29 July 2025; accepted 4 September 2025. Date of publication 8 September 2025; date of current version 2 December 2025. This work was supported by the National Key Research and Development Program under Grant 2022YFB2901202. Recommended for acceptance by Dr. Dusit Niyato. *(Corresponding author: Hao Jiang.)*

The authors are with the Electronic Information School, Wuhan University, Wuhan 430072, China (e-mail: eehaoli@whu.edu.cn; jh@whu.edu.cn; wujing@whu.edu.cn; whu_guiao@whu.edu.cn; zhangjian@whu.edu.cn).

Digital Object Identifier 10.1109/TNSE.2025.3607331

2327-4697 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Bandwidth limitations: Limited network bandwidth restricts the speed of gradient synchronization between accelerators within or across data centers, resulting in prolonged training times and reduced overall efficiency.

Accelerator idling: During gradient synchronization, accelerators remain idle while waiting for gradients from other accelerators, leading to wasted computing resources and increased training costs [20].

Therefore, reducing the impact of gradient synchronization on the efficiency of distributed LLM training across multiple data centers is of paramount importance [21]. Many schemes [22], [23] have been proposed to decrease both the size and synchronization frequency of gradients. These solutions often combine various forms of parallelism, such as data, tensor, or expert parallelism [21], [24], with algorithmic optimizations in gradient synchronization [17], [25] [26]. Although these techniques are effective in improving communication efficiency, their overall performance improvement is still constrained by the available network bandwidth [18]. This limitation is particularly pronounced in scenarios involving communication across multiple data centers, where bandwidth heterogeneity and contention further exacerbate synchronization latency.

To overcome these challenges, more advanced methods such as branch-train-merge (BTM) [27] and branch-train-mix (BTX) [28] have been proposed, introducing architectural innovations for the design and training of LLMs. These methods divide LLM training into two stages. In the first stage, multiple expert LLMs are independently trained on datasets from different domains. In the second stage, all expert LLMs are merged into a unified model using the mixture of experts (MoE) [29] architecture or model merging techniques [30]. The traditional method trains each expert LLM independently in a single data center. However, this method has the following problems: (1) Gradient synchronization latency within the data center continues to reduce the training efficiency. (2) Heterogeneity in computational and storage resources across data centers can lead to unbalanced training convergence progress among expert LLMs. As a result, some data centers may become idle, resulting in underutilization of computational resources. Although training the same expert LLM across multiple data centers introduces additional gradient synchronization latency, this overhead can be mitigated by parallelizing gradient computation and synchronization, thereby enhancing the overall efficiency of distributed LLM training [31].

In this work, we propose an effective distributed training scheme called PGCS for LLMs. Specifically, the scheme addresses the above challenge with the following designs: (1) *Local batch size allocation:* PGCS adjusts the local batch size assigned to each data center based on its computing or communication resources, allowing resource-constrained data centers to train appropriately scaled data. (2) *Expert structure uniformity:* All expert models share the same structure, enabling switching between different experts by updating parameters without the need to load a new expert model. (3) *Parallel gradient computation and synchronization:* Training efficiency can be enhanced by parallelizing the gradient synchronization of one expert model with the gradient computation of another. (4) *Synchronization latency optimization:* To minimize the gradient synchronization latency, we formulate the gradient synchronization process and design an algorithm to find the optimal gradient synchronization strategy.

The main contributions of this work are summarized as follows:

- An effective distributed training scheme for LLMs named PGCS is proposed to enhance the efficiency of distributed training by parallelizing the processes of gradient computation and gradient synchronization.
- A gradient synchronization algorithm named BLP is developed to find the optimal gradient synchronization for arbitrary network connectivity and limited bandwidth across multiple data centers.
- The efficiency of PGCS and BLP is validated by conducting simulations and experiments with multiple gradient synchronization algorithms and network topologies.

The remainder of this paper is organized as follows. Section II reviews related research on enhancing the training efficiency of LLMs. Section III describes the system architecture, formalizes the problem, and presents the methods proposed in this paper. Section IV analyzes the performance evaluation results. Finally, Section V provides a summary and an outlook for future work.

II. Related Work

We provide a review of related work from five perspectives. A detailed comparison between existing methods and our approach is summarized in Table I.

A. Parallel Training

Traditional distributed training of LLMs requires maintaining multiple copies of LLM on different GPUs and strict synchronization. Therefore, the communication latency introduced by frequent synchronization becomes a key bottleneck. To mitigate this issue, recent studies have explored parallel training strategies based on expert models. For example, BTM [27] trained multiple expert LLMs in parallel, each customized for a specific domain. Since the training of the expert model is independent, this method can greatly reduce the communication volume. Similarly, BTX [28] and BM [32] adopt MoE and Arcee Fusion [30] frameworks to merge expert models into a unified architecture. During the parallel training of expert LLMs, the communication latency can be reduced because there is no need to synchronize any parameters between expert LLMs. However, these methods did not consider how to improve training efficiency when multiple expert LLMs were trained simultaneously.

B. Model Merging

Model merging refers to the process of combining multiple models into a single unified model [41]. This technique generally falls into two categories: merging models with identical architectures and merging models with different architectures. The former typically leverages mode connectivity [42] to enable

TABLE I
COMPARISON BETWEEN EXISTING METHODS AND OUR APPROACH

Method Category	Representative Works	Key Contributions	Limitations	Differences from Our Work
Parallel Training	BTM [27], BTX [28], BM [32]	Train multiple expert LLMs in parallel	Lack of strategies to improve efficiency of parallel expert training	Focus on efficiency improvement for parallel expert training
Model Merging	CALM [33], FUSELLM [34]	Merge expert models with different architectures	Require additional training, no true architectural integration	Use structurally identical expert models
Parallelism Approaches	Jointflow [35], Tesseract [36], Gpipe [37], Mpmoe [38]	Improve training efficiency within or between data centers	Sensitivity to bandwidth and heterogeneity	Data categorization and local batch allocation
Gradient Compression	Mipd [22], Fedcspc [39],ACCO [31], Gaia [19]	Reduce gradient size	Degrade model quality	Parallel gradient computation and synchronization
Gradient Synchronization	BlueConnect [25], Wrht [26], FlexReduce [40], Aggregation tree [18]	Algorithmic optimization	Separate optimization for latency within or between data centers	Unified optimization for latency within and between data centers

efficient merging via linear or non-linear interpolation in parameter space [43], [44]. The latter, while more flexible in accommodating models with different initializations or structures, often requires additional training and cannot achieve true architectural integration [33], [34]. Therefore, adopting expert models with identical architecture is not only a reasonable choice in our work, but also a necessary one to effectively apply techniques such as task vectors and interpolation for model merging.

C. Parallelism Approaches

To enhance the efficiency of distributed training for LLMs across multiple data centers, several parallelization approaches can be employed, including data parallelism [45], [46], model parallelism [35], [47], tensor parallelism [36], [48], and pipeline parallelism [37], [49], [50]. The choice of parallelization approaches includes two parts: within each data center and between multiple data centers. Within each data center, a hybrid approach that combines several forms of parallelism should be used depending on the type and number of accelerators, as well as the network connectivity and bandwidth [51]. Due to the need for frequent communication in model and tensor parallelism, they are not suitable for the limited bandwidth environment between data centers. Instead, data and pipeline parallelism are more feasible [21], [24], with data parallelism generally preferable due to its better tolerance to computational and network heterogeneity. Pipeline parallelism requires a strict sequential flow of activations and gradients at every stage, and any imbalance in the availability of computing or communication resources between data centers may lead to significant pipeline bubbles [38]. In contrast, data parallelism allows each data center to train different input samples independently and only requires synchronization at periodic intervals. By adapting the mini-batch size for each data center according to its computational power and memory capacity, data parallelism can be more adaptable to the heterogeneity of computing resources and more suitable for scenarios with limited interconnection bandwidth.

D. Gradient Compression Techniques

Some studies [22], [23] [39] have focused on reducing the size of gradients through techniques such as gradient sparsity—where only the most significant gradients are transmitted—and gradient quantization, which reduces the precision of gradients. Additionally, techniques such as gradient accumulation have been proposed to reduce the frequency of synchronization [31]. Gaia [19] only transfers the gradients of important parameters between data centers and decouples the communication process within the data centers from the communication process between data centers. LaVFL [52] proposes a layer-by-layer verification mechanism combined with convolution gradient compression to reduce communication overhead. SwitchATC [53] adopts a parameter-server architecture and improves overall training performance by discarding sparse gradients while retaining dense gradients during synchronization. However, these approaches have two major limitations: they can degrade model quality, and the enhancement in training efficiency is limited by the network bandwidth.

E. Gradient Synchronization Algorithms

In addition to the above studies, the gradient synchronization latency can be reduced by optimizing the gradient synchronization algorithm. The gradient synchronization latency consists of two parts: latency within the data center and latency between data centers. Much research has been devoted to optimizing gradient synchronization algorithms within data centers, including BlueConnect [25], Wrht [26], and FlexReduce [40]. The study in [54] proposed the near-optimal forest packing algorithm to maximally utilize available bandwidth and parallelized reduction and broadcast stages to minimize gradient synchronization latency between data centers. Some researchers [17], [18] utilized high-speed reconfigurable optical networks and proposed scheduling algorithms to accelerate gradient synchronization. MoESys [55] optimizes the training process of the distributed mixture of experts model, improving the efficiency of gradient communication and memory utilization. In federated learning scenarios for distributed LLMs, FedMRG [56] enhances gradient communication efficiency through the use of low-rank adaptation and a dual-adapter mutual boosting module, significantly reducing the amount of information exchanged while maintaining model quality. Furthermore, frameworks such as DeepSpeed and Megatron-LM [57] not only reduce communication overhead but also exploit the overlap between computation and communication to enhance training efficiency. The above

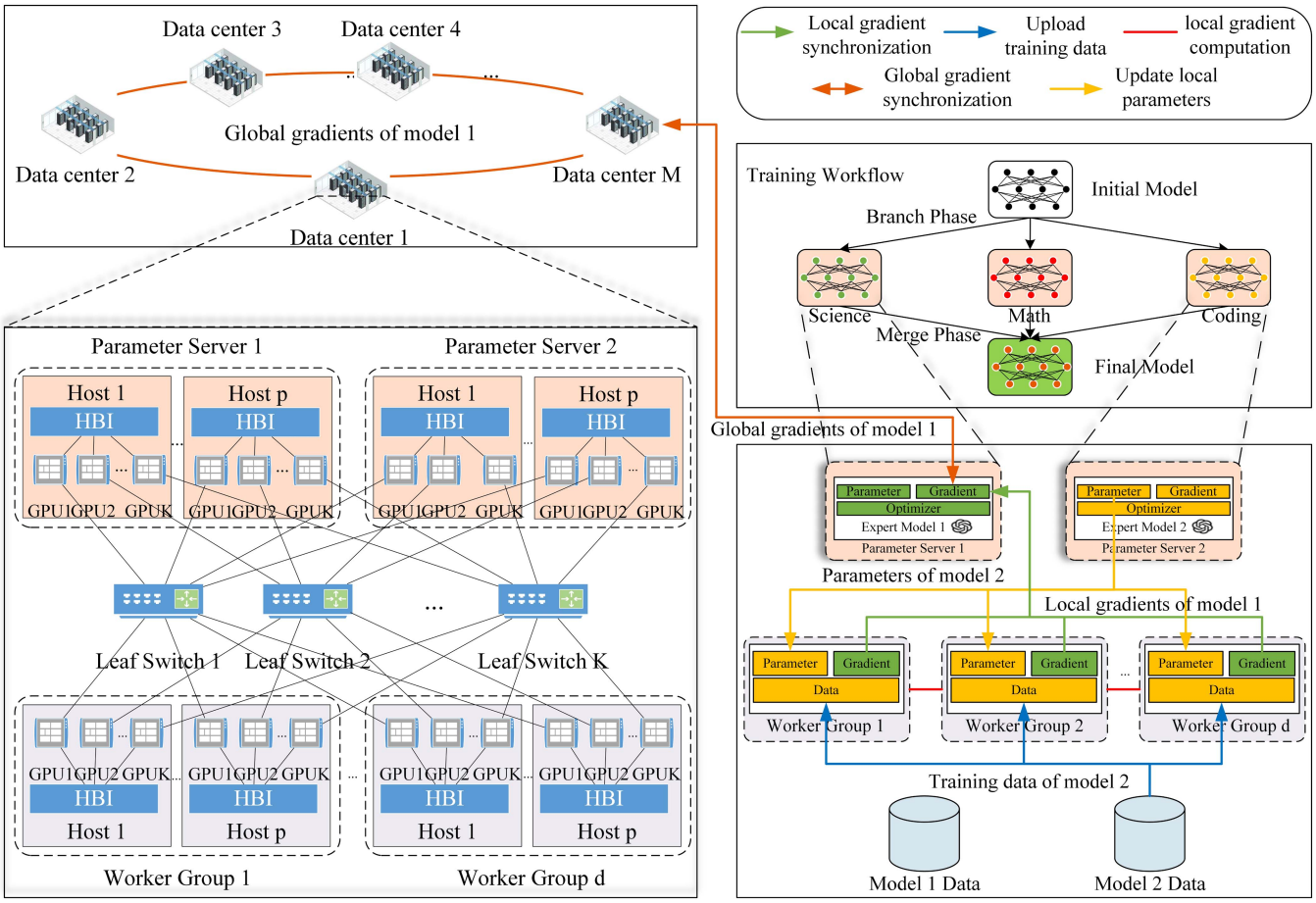


Fig. 1. Overview of the system architecture for PGCS.

approaches focus only on optimizing gradient synchronization either within data centers or between them, treating these as distinct challenges. In contrast, our research integrates latency within and between data centers into a unified optimization framework, with the goal of minimizing gradient synchronization latency.

III. Methodology

This section introduces the system architecture of PGCS and formulates the gradient synchronization latency problem. To address it, an algorithm that integrates the bisection method with linear programming is proposed to determine the optimal solution.

A. System Architecture

An overview of the PGCS system architecture is shown in Fig. 1. This architecture integrates a cost-efficient, high-performance network design with advanced parallelism approaches and gradient synchronization algorithms to enable efficient training of LLMs.

Network Architecture: The network architecture includes connections within and between data centers. For efficient training across geographically distributed data centers, our scheme uses optical networks between data centers, offering low cost, large capacity, and high speed to support the data volumes needed for gradient synchronization [58]. Within each data center, we use a Rail-only architecture [59], a cost-effective alternative to traditional high-performance networks. This architecture removes the expensive spine layer while still maintaining performance comparable to that of the state-of-the-art GPU networks.

Data Categorization: The overall training dataset is divided into multiple subsets, each corresponding to a specific domain. When all data centers load the same expert model and train on the same specialized dataset, we ensure that all data centers participate in training the same expert model in each round.

Local Batch Allocation: Considering the heterogeneity in computational power and memory capacity between data centers, we adjust the local batch size assigned to each data center accordingly. This strategy ensures that all participating data centers complete their gradient computations concurrently at each training step, thus improving synchronization efficiency.

Training Workflow: In our framework, we first independently train multiple expert models on specific datasets using a unified initial LLM. Each expert model is specialized for a particular domain, such as science, mathematics, or coding. These expert models can subsequently be merged into a unified LLM. However, the focus of this work is not on the merging process, but on the efficient training of expert models across multiple data centers. This uniform structure allows switching between expert

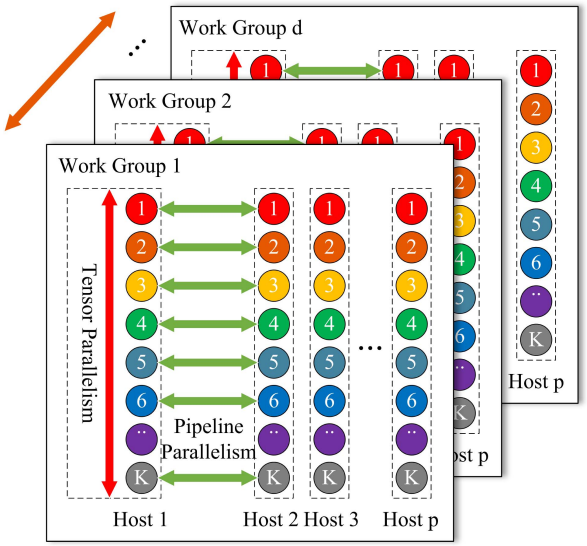


Fig. 2. Parallelism strategy of PGCS within the data center.

models by replacing parameters of the current expert model, without the need to reload a new expert model.

Parallelism Strategy: To efficiently train LLMs, PGCS combines data parallelism, pipeline parallelism, and tensor parallelism within each data center. The GPUs in a data center are organized into two main roles: worker groups and parameter servers. The worker groups are responsible for gradient computation, while the parameter servers handle gradient synchronization and distribute the updated parameters. Each worker group maintains a complete copy of the model parameters and processes distinct partitions of the training data in parallel with other worker groups. Within each worker group, the LLM is divided into multiple stages, with each stage mapped to a different host. The training process is accelerated by using pipeline parallelism between hosts and tensor parallelism within hosts. Fig. 2 illustrates the parallelism strategy employed by PGCS.

Memory Optimization: Traditional training schemes require each GPU to store the parameters of the LLM, along with the corresponding gradients and optimizer states. This approach places significant memory pressure on GPUs, especially when training larger models, as the memory required for storing both the model and its associated metadata increases with model size. Our scheme implements a memory optimization strategy by offloading gradient synchronization and parameter update to parameter servers. Worker groups store only essential parameters and gradients, significantly reducing memory overhead on individual GPUs.

Gradient Synchronization: In our scheme, gradient synchronization occurs in two stages: local gradient synchronization within each data center and global gradient synchronization across data centers.

Local Gradient Synchronization: Within each data center, worker groups compute gradients independently. Once gradient computation is complete, the parameter servers aggregate the gradients from all worker groups.

TABLE II
LIST OF MAIN NOTATIONS

Notation	Description
S	size of the gradient for the model
$\mathcal{R}$	set of directed spanning trees
$\mathcal{B}$	set of reverse spanning trees
$\mathcal{R}'$	subset of directed spanning trees
r^x	the directed spanning tree r^x
C^x	size of gradient reduced/broadcast by tree r^x
$\mathcal{P}^x$	set of paths from the leaf node to root node in tree r^x
H_l^x	bandwidth of link l utilized by tree r^x
Q_k^x	effective bandwidth of path k
d_l	propagation delay of link l
T_l	gradient synchronization latency within data centers
T_g	gradient synchronization latency across data centers
B_l	available bandwidth of link l

Global Gradient Synchronization: After local gradient synchronization, parameter servers across different data centers perform global gradient synchronization.

Parameter Synchronization: After global gradient synchronization, the parameter server updates the model parameters based on the global gradient. As the model proceeds to the next training round, the parameter server distributes updated parameters to its local worker groups. Since the global gradient is the same across all data centers, this process ensures that model parameters remain consistent across all data centers.

To improve training efficiency, this work proposes a parallelization scheme for gradient computation and synchronization. An example of training two expert models is illustrated in Fig. 1. We predefine a fixed training order of expert models (e.g., Expert 1 and Expert 2 are trained in an alternating manner). Specifically, after expert model 1 has completed the gradient computation, it simultaneously performs local gradient synchronization for itself and global parameter distribution for expert model 2. Subsequently, during the training of expert model 2, parameter server 1 is responsible for synchronizing gradients for expert model 1. This parallelization allows computation and communication to occur simultaneously, reducing system idle time and enhancing training efficiency. When the gradient synchronization latency of one expert model is fully overlapped by the gradient computation time of another, PGCS can maximize the training efficiency of LLMs. Thus, the primary optimization goal is to minimize gradient synchronization latency, which in turn accelerates the training of LLMs.

B. Problem Formulation

The optimization process is divided into two stages: local gradient synchronization and global gradient synchronization. For clarity, the main notations used in this work are summarized in Table II.

1) Local Gradient Synchronization: Gradient synchronization within the data center involves performing a reduce operation to aggregate gradients from all worker groups to the parameter servers. Due to the data parallelism employed across worker groups, the gradient data generated by each worker

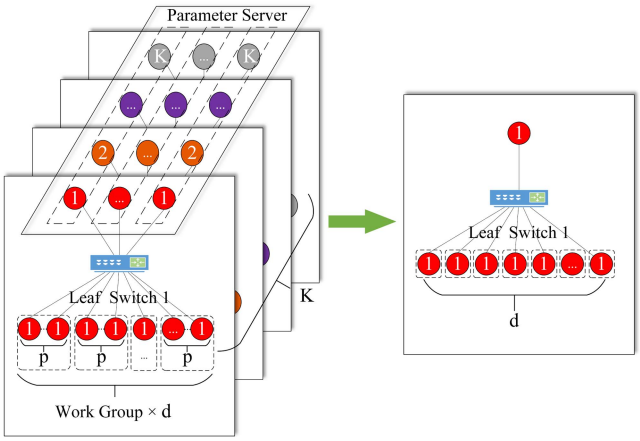


Fig. 3. Abstraction of the topology within the data center during local gradient synchronization.

group has the same size, dimensions, and structure. Furthermore, owing to pipeline parallelism within each worker group, each host stores gradients corresponding to different parameters. Additionally, since tensor parallelism is employed on each host, the gradient data on all GPUs within the same host is identical. Based on the above analysis, if the total gradient size for the entire model is denoted by S, the gradient size per host is $\frac{S}{p}$. Thus, the gradient synchronization latency problem within data centers can be simplified as follows: consider a leaf switch connecting $d+1$ GPUs, with gradients on these GPUs corresponding to the same set of parameters. Among these GPUs, d are from different worker groups, and the remaining one belongs to the parameter server, as illustrated in Fig. 3. The goal is to minimize the time required to reduce gradients from all worker groups to the parameter server. According to Lemma 1, the gradient synchronization latency within a data center can be approximated as

$$T_l = \frac{S}{B_s p}, \tag{1}$$

where B_s is the bandwidth between the switch and the GPU.

Lemma 1: Considering only the transmission delay, assume that the gradient size per GPU is S/p and that the bandwidth of each link is B_s. The minimum time required to reduce all gradients is asymptotically $\frac{S}{B_s p}$.

Proof: The gradients on each GPU are divided into m equal parts. These parts from the first GPU are transmitted sequentially to the second GPU, where each received part is immediately merged with its corresponding local part. Once merged, the result is sent to the third GPU. This process continues until all parts are eventually aggregated on the $d+1$-th GPU. The time taken for the first part from the first GPU to merge with the $(d+1)$-th GPU is $\frac{2Sd}{B_s mp}$, while the time interval for transferring subsequent parts between GPUs is $\frac{S}{B_s mp}$. Thus, the total time for the reduction process is $t = \frac{S}{B_s p} + \frac{(2d-1)S}{B_s mp}$. Given that $m \gg d$, the time can be approximated as $\frac{S}{B_s p}$.

2) Global Gradient Synchronization: Unlike the reduce operation within a single data center, gradient synchronization across data centers involves two distinct operations: gradient reduction and gradient broadcast. Specifically, all gradients from the parameter servers must first be aggregated through reduce operation, followed by broadcast to all parameter servers. Two characteristics of gradients are crucial for accelerating this process: (1) gradients can be partitioned, and (2) their size remains constant after reduction. An optimized communication method is essential for efficient inter-data center bandwidth utilization and to avoid redundant transmissions. This work proposes a spanning tree-based communication structure that constructs multiple gradient reduction/broadcast trees, with each tree responsible for reducing or broadcasting a portion of the gradients in parallel. By optimizing the size of gradients synchronized within each tree and the bandwidth of all physical links along them, communication overhead and gradient synchronization latency are minimized.

The connections between data centers are modeled as a graph $G = (V, E)$, where V is the set of data centers, and each link $l \in E$ represents a connection between two data centers with d_l as the propagation delay of link l. The set of directed spanning trees and reverse spanning trees in this graph are denoted as $\mathcal{R}$ and $\mathcal{B}$, respectively. For each tree $r^x \in \mathcal{R}$, the variable C^x represents the size of gradients reduced on that tree, while H_l^x denotes the bandwidth of link l utilized by tree r^x. Additionally, the set of paths from each leaf node to the root in tree r^x is denoted by $\mathcal{P}^x$. Since the data transmission rate along a path is constrained by the bandwidth of the bottleneck link, the effective bandwidth Q_k^x of path $k \in \mathcal{P}^x$ is given by

$$Q_k^x = \min_{l \in k} H_l^x. \tag{2}$$

Lemma 2: Neglecting the processing time for reducing or broadcasting gradients and considering only the transmission and propagation delays, the time T_k^x for reducing or broadcasting gradients of size C^x along path k in tree r^x is given by

$$T_k^x = \frac{C^x}{Q_k^x} + \sum_{l \in k} d_l. \tag{3}$$

Proof: When the gradient of size C^x is split into m equal parts, the transmission delay for the first part is given by $\frac{C^x}{m \cdot Q_k^x}$, where Q_k^x is the effective bandwidth of path k. Consider that path k traverses M data centers. The time for the first part of gradients to reach the M-th data center is $\frac{M-1}{m} \cdot \frac{C^x}{Q_k^x} + \sum_{l \in k} d_l$, where $\sum_{l \in k} d_l$ represents the total propagation delay across all physical links in path k. The time for the last part of the gradients to reach the M-th data center is $\frac{M+m-2}{m} \cdot \frac{C^x}{Q_k^x} + \sum_{l \in k} d_l$. Since $m \gg M$, we have $\frac{M+m-2}{m} \approx 1$. Therefore, the time for reducing or broadcasting gradients of size C^x along path k in tree r^x can be simplified to $\frac{C^x}{Q_k^x} + \sum_{l \in k} d_l$.

The objective is to minimize the gradient synchronization latency across data centers. Since gradient reduction and gradient broadcast are inverse processes, the total gradient synchronization latency is twice the reduction time. An illustration is provided in Fig. 4. When the values of S, B_s, and p are known, the gradient synchronization latency within a data center T_l becomes a fixed constant. By eliminating all constant terms

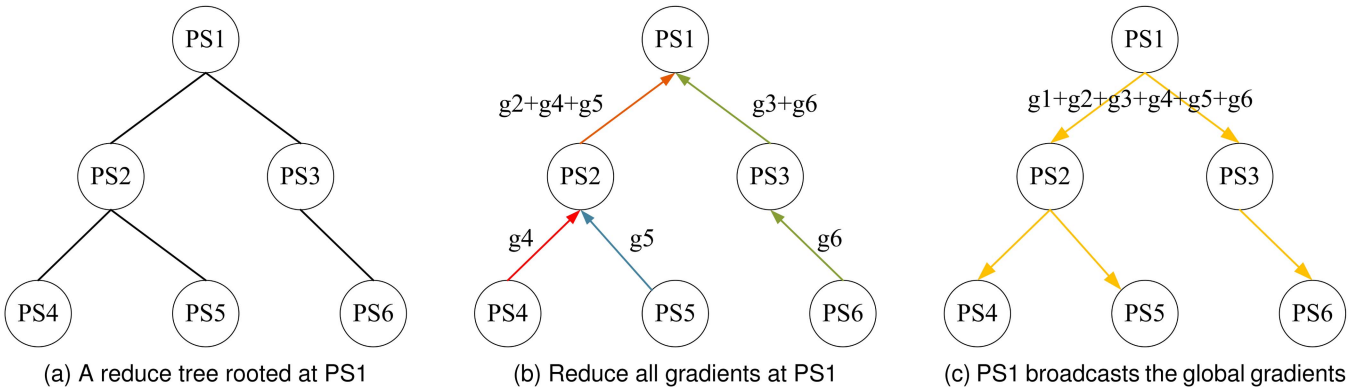


Fig. 4. Gradient reduction and gradient broadcast: an example with 6 data centers.

in the model, the problem can be formally expressed as follows

$$(\mathbf{P1}): \min_{w^x, C^x, H_l^x} \quad T_g \tag{4a}$$

$$\text{s.t.} \quad S = \sum_{r^x \in \mathcal{R}} w^x C^x, \tag{4b}$$

$$w^x \left(\frac{C^x}{Q_k^x} + \sum_{l \in k} d_l \right) \le T_g,$$

$$\forall r^x \in \mathcal{R}, \forall k \in \mathcal{P}^x,$$

$$\sum_{r^x \in \mathcal{R}} \sum_{l \in r^x} w^x H_l^x \le B_l, \quad \forall l \in E,$$

$$H_l^x > 0, \quad C^x > 0, \quad w^x \in \{0, 1\}, \tag{4c}$$

where the variable $w^x = 1$ indicates that the tree r^x is selected. Constraints in (4b) ensure that the gradients from all data centers are synchronized. According to Lemma 2, the latency along path k in tree r^x is $\frac{C^x}{Q_k^x} + \sum_{l \in k} d_l$. Constraints in (4c) specify that the latency along each path must not exceed the maximum time, ensuring optimality of the solution. Similarly, constraints in (4d) ensure that the total bandwidth used on each link does not exceed the available capacity. Finally, constraints in (4e) impose non-negativity restrictions, requiring that both variables H_l^x and C^x remain greater than zero.

C. Proposed Algorithms

Problem (**P1**) can be divided into two sub-problems: finding the optimal set of trees and optimizing the size of gradients synchronized within each tree and the bandwidth of all physical links along each tree. Given the limited number of data centers and their simple connections, the optimal solution can be determined by comparing gradient synchronization latency across all possible sets of spanning trees. Thus, problem (**P1**) can be simplified as follows:

$$(\mathbf{P2}): \min_{C^x, H_l^x} \quad T_g \tag{5a}$$

$$\text{s.t.} \quad S = \sum_{r^x \in \mathcal{R}'} C^x, \tag{5b}$$

$$\frac{C^x}{Q_k^x} + \sum_{l \in k} d_l \le T_g, \quad \forall r^x \in \mathcal{R}', \forall k \in \mathcal{P}^x, \tag{5c}$$

$$\sum_{r^x \in \mathcal{R}'} \sum_{l \in r^x} H_l^x \le B_l, \quad \forall l \in E, \tag{5d}$$

$$H_l^x > 0, \quad C^x > 0, \tag{5e}$$

where a subset of $\mathcal{R}$ is denoted by $\mathcal{R}'$.

By solving problem (**P2**), the original problem (**P1**) is addressed as shown in Algorithm 1. An algorithm called BLP, which combines the bisection method with linear programming (LP), is proposed. The bisection method is used to progressively narrow the feasible range of T_g, while LP is employed at each iteration to verify the existence of a feasible solution under the current T_g. Specifically, the algorithm begins by initializing the lower and upper bounds, T_{lb} and T_{ub}, respectively. The algorithm then iterates through all subsets of spanning trees $\mathcal{R}' \subseteq \mathcal{R}$. For each subset, LP is used to verify the existence of a feasible solution, and the bounds are updated accordingly. The procedure continues until the optimal synchronization time T_g is determined.

Lemma 3: When a solution $\{C^x\}, \{H_l^x\}$, satisfies all constraints and the delay along path k satisfies $\frac{C^x}{Q_k^x} + \sum_{l \in k} d_l < T_g$, then there exists another solution $\{C^x\}, \{H'^x_l\}$ such that the delay along all paths equals T_g.

Proof: For path k, the delay satisfies $\frac{C^x}{Q_k^x} + \sum_{l \in k} d_l < T_g$, meaning path k has been allocated more bandwidth than necessary. To make efficient use of the available bandwidth, the new effective bandwidth Q'^x_k for path k is given by

$$Q'^x_k = \frac{C^x}{T_g - \sum_{l \in k} d_l} \le Q_k^x. \tag{6}$$

The bandwidth H'^x_l of link l, utilized by tree r^x, is given by

$$H'^x_l = \max_{k \in \mathcal{P}^x, l \in k} Q'^x_k. \tag{7}$$

For path k, $Q'^x_k \le Q_k^x$, so the bandwidth constraint is satisfied:

$$\sum_{r^x \in \mathcal{R}'} \sum_{l \in r^x} H'^x_l \le B_l, \quad \forall l \in E. \tag{8}$$

Algorithm 1: BLP.

Input: S, B_l, N: number of iterations.
Output: T_g.
$T_{lb} = 0$.
$T_{ub} = \frac{S}{B_l}$.
for each $\mathcal{R}' \subseteq \mathcal{R}$ **do**
  if $T_g = T_{lb}$, $\exists X$ that satisfies equation (15) **then**
    $T_{ub} = T_{lb}$,
    $T_{lb} = 0$,
    $T_g = (T_{lb} + T_{ub})/2$,
    Update T_{ub} and T_{lb} by solving problem (**P3**) using Algorithm 2
  else if $T_g = T_{ub}$, $\nexists X$ that satisfies equation (15) **then**
    continue
  else
    $T_g = (T_{lb} + T_{ub})/2$,
    Update T_{ub} and T_{lb} by solving problem (**P3**) using Algorithm 2
  end if
end for
$T_g = T_{ub}$.
return

Since $T_g > \sum_{l\in k} d_l$, the constraint $Q'^x_k > 0$ is automatically satisfied. Hence, there exists another solution $\{C^x\}, \{H'^x_l\}$ such that the delay along all paths equals T_g.

To simplify problem (**P2**), let W^x represent the effective bandwidth of tree r^x. The effective bandwidth of path k in tree r^x is given by

$$Q'^x_k = \frac{C^x W^x}{C^x - W^x \sum\limits_{l\in k} d_l}, \quad \forall r^x \in \mathcal{R}', \forall k \in \mathcal{P}^x. \tag{9}$$

According to Lemma 3, the size of the gradients C^x reduced or broadcast in tree r^x is given by

$$C^x = \frac{S \cdot W^x}{\sum\limits_{r^x\in\mathcal{R}'} W^x}. \tag{10}$$

Substituting equation (10) into equation (9), the effective bandwidth of path k in tree r^x is given by

$$Q'^x_k = \frac{S \cdot W^x}{S - \sum\limits_{r^x\in\mathcal{R}'} W^x \cdot \sum\limits_{l\in k} d_l}, \forall r^x \in \mathcal{R}', \forall k \in \mathcal{P}^x. \tag{11}$$

Thus, the dual problem of (**P2**) is as follows:

$$(\mathbf{P3}): \min_{W^x} \quad T_g \tag{12a}$$

$$\text{s.t.} \quad T_g = \frac{S}{\sum\limits_{r^x\in\mathcal{R}'} W^x}, \tag{12b}$$

$$H'^x_l = \max_{k\in\mathcal{P}^x, l\in k} \frac{S \cdot W^x}{S - \sum\limits_{r^x\in\mathcal{R}'} W^x \cdot \sum\limits_{l\in k} d_l}, \tag{12c}$$

$$\sum_{r^x\in\mathcal{R}'} \sum_{l\in r^x} H'^x_l \le B_l, \quad \forall l \in E, \tag{12d}$$

$$W^x > 0. \tag{12e}$$

Algorithm 2: Solving Problem (**P3**) by LP.

Input: T_{lb}, T_{ub}, T_g, N.
Output: T_{lb}, T_{ub}, T_g.
$n = 0$.
while $n < N$ and $T_{ub} - T_{lb} > \varepsilon$ **do**
  if $\exists X$ that satisfies equation (15) **then**
    $T_{ub} = T_g$,
    $T_g = (T_{lb} + T_{ub})/2$
  else
    $T_{lb} = T_g$,
    $T_g = (T_{lb} + T_{ub})/2$
  end if
  $n = n + 1$.
end while
return

Lemma 4: Given a fix value of T_g, all constraints of problem (**P3**) are linear.

Proof: When T_g is treated as a constant, constraint (12b) can be rearranged as follows:

$$\sum_{r^x\in\mathcal{R}'} W^x = \frac{S}{T_g}, \tag{13}$$

which is a linear equality constraint with respect to W^x.

Next, consider constraint (12c), which is equivalent to the following:

$$H'^x_l = \max_{k\in\mathcal{P}^x, l\in k} \frac{T_g \cdot W^x}{T_g - \sum\limits_{l\in k} d_l}. \tag{14}$$

In this expression, H'^x_l is linearly dependent on W^x because the denominator $T_g - \sum_{l\in k} d_l$ is constant when T_g is fixed. Although the maximum operator is applied, it merely selects the largest value among linear terms, which does not affect the linearity of each individual term. Therefore, H'^x_l remains a linear function of W^x.

Since H'^x_l is linearly dependent on W^x, constraint (12d) can be expressed as a linear inequality, ensuring that the sum of the effective bandwidths H'^x_l across all paths and spanning trees does not exceed the available link bandwidth B_l. Finally, constraint (12e) is a straightforward non-negativity linear constraint. Thus, problem (**P3**) is a linear programming problem when T_g is fixed, as all constraints are linear.

According to Lemma 4, problem (**P3**) is equivalent to:

$$\begin{cases} \hat{\mathbf{u}} \cdot X = \frac{S}{T_g} \\ AX \le B_l \hat{\mathbf{u}}^T \\ X > 0 \end{cases}, \tag{15}$$

where $X = [W^1, W^2, \ldots, W^x]^T$, $A \in \mathbb{R}^{l\times x}_{\ge 0}$, $\hat{\mathbf{u}}$ represents the unit row vector. Equation (15) can be solved efficiently using linear programming. The procedure for solving problem (**P3**) is summarized in Algorithm 2. Specifically, the bisection method halves the current search interval for T_g and computes the midpoint. At each iteration, linear programming is utilized to verify whether a feasible set of variables X satisfies all constraints for the current T_g. If a feasible solution exists, the upper bound of

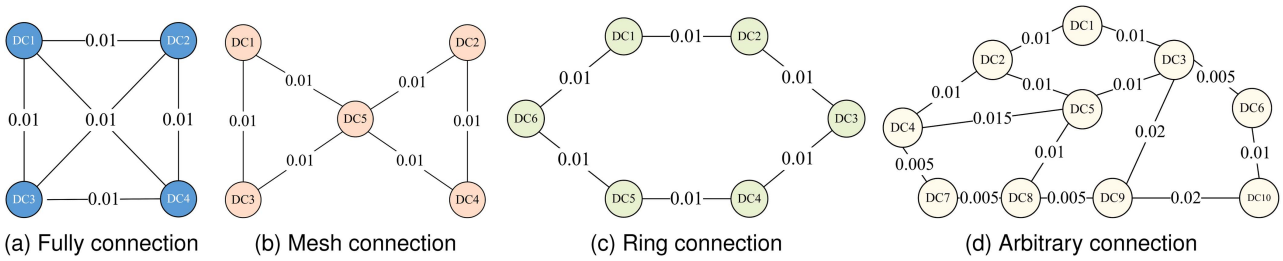


Fig. 5. The connections and link propagation delay between data centers.

the interval is updated; otherwise, the lower bound is adjusted. This process continues until the difference between the upper and lower bounds falls below a predefined threshold ε, ensuring convergence to the optimal solution.

IV. Performance Evalution

To comprehensively evaluate the proposed method, we divide the experiments into two parts: large-scale simulation analysis and small-scale physical experiments. The simulations are conducted to analyze performance trends under different data center scales and network topologies, while the physical experiments are designed to demonstrate the feasibility and stability of the approach in practical deployment environments.

A. Simulation Analysis

1) Simulation Setup: This subsection describes the simulation environment used to evaluate the performance of PGCS and BLP.

Data center: To evaluate the effectiveness of PGCS and BLP, we simulate a distributed training environment consisting of multiple data centers. In real-world scenarios, data centers are equipped with varying computational capacities. During the training of LLMs, if data centers with different computational capacities are assigned training tasks of equal size, those with higher computational capacities will remain idle while waiting for the slower centers to finish gradient computations, leading to significant inefficiencies. To mitigate this, we assign smaller tasks to data centers with lower computational capacities, ensuring that all centers complete gradient computations simultaneously. For simplicity, the equivalent computational capacity of all data centers is 10^{18} floating point operations per second (FLOPS).

Connection: Within each data center, a Rail-only architecture is used. The connections between data centers and link propagation delay are configured as shown in Fig. 5. To be specific, the following connections are applied: 4 data centers are fully connected, 5 data centers are organized in a mesh topology, 6 data centers are arranged in a ring topology, and 10 data centers use arbitrary connections. These topologies are analyzed to evaluate their impact on the performance of the proposed schemes.

Bandwidth: The link bandwidth within each data center B_s is set to 1 Tbps. To emulate the dynamic nature of the network between data centers, the available bandwidth B_l of all links between data centers is limited to a range of 500 (Gbps) to 1 (Tbps).

Model: We employ one LLM with 1 trillion parameters as the initial expert model and train ten independent expert models based on this. Each expert is trained separately on data in specific fields. After the training of all expert models is completed, these expert models are merged into a unified model through model aggregation. Notably, the focus of this work is on the efficient training of expert models across multiple data centers, rather than model merging techniques.

Dataset: We assume a total dataset size of 10 T tokens, with each expert model trained on 1 T tokens. To ensure utilization of computational resources, the dataset is evenly partitioned across all data centers. The batch size per training step is set to 10^7 tokens.

Comparing baselines: To demonstrate the advantages of the proposed schemes, we compare them against the following baselines:

- *PS:* A traditional synchronous training approach where gradients are aggregated on central parameter servers.
- *Ring Allreduce:* A decentralized gradient synchronization method where nodes communicate in a ring topology to reduce communication bottlenecks.
- *Aggregation tree* [18]: A topology-adaptive parameter synchronization approach that organizes nodes in a tree structure to optimize communication efficiency.

Performance metrics: We evaluate the performance of the proposed schemes using the following metrics:

- *Total training time:* The overall duration required to train the model. It directly reflects the efficiency of the training process. Reducing the total training time is crucial because it not only saves computational resources but also accelerates the model development cycle.
- *Gradient Synchronization Latency:* The time required for synchronizing gradients during each training step. We measured the gradient synchronization latency under different network connections and bandwidths.
- *Training loss:* The training loss is the difference between model predictions and true values. The rate at which the training loss decreases indicates the convergence speed of the model during training.

2) Evaluation Models Based on Scaling Laws: To accurately evaluate the efficiency of our proposed schemes, we develop evaluation models based on scaling laws [60]. This model quantifies the relationship between model size, computational resources and network characteristics and training efficiency.

*Computation Evaluation Model:*According to the scaling laws, the estimate of floating point operations (FLOPs) C required for the total training is given by

$$C \approx \lambda N_m N_b N_s, \tag{16}$$

where λ represents the average number of FLOPs per parameter per token during the forward and backward passes, N_m indicates the number of model parameters, N_b is the batch size per training step, and the number of training steps is denoted by N_s. Typically, λ is set to 6, as each parameter in a neural network participates in multiple FLOPs during both the forward and backward passes. Specifically, the forward pass requires one multiplication and one addition, and the backward pass requires two multiplications and two additions, leading to a total of 6 FLOPs per parameter per token.

We assume that the computational capacity of each data center is F FLOPS, then the computation time per training step T_c in each data center is given by

$$T_c = \frac{C}{N_d N_s F}, \tag{17}$$

where N_d represents the number of data centers. When the total dataset size and the batch size are D and N_b, the total number of training steps N_s is given by

$$N_s = \frac{D}{N_b}. \tag{18}$$

*Communication Evaluation Model:*In our communication evaluation model, the gradient synchronization latency for the i-th training step is given by

$$T_s^i = 2\left(T_l + T_g^i\right). \tag{19}$$

Here, local gradient synchronization latency T_l has been given by equation (1), and global gradient synchronization latency T_g^i of the PS and Ring Allreduce is as follows.

In the PS architecture, each data center communicates directly with a central parameter server. The gradient synchronization latency is given by

$$T_{\text{PS}} = \frac{S}{B_a} + L_a, \tag{20}$$

where S represents the gradient size of the model, the average bandwidth between data centers is denoted by B_a, and L_a indicates the link propagation delay between data centers.

In Ring Allreduce, data centers are arranged in a ring topology. The gradient synchronization latency is given by

$$T_{\text{Ring}} = (N_d - 1)\left(\frac{S}{N_d B_a} + L_a\right). \tag{21}$$

3) Results and Analysis: In this subsection, we analyze the impact of PGCS and BLP on training efficiency, focusing on total training time, gradient synchronization latency, and training loss.

Total training time: For traditional synchronization schemes, the total training time is defined as

$$T_{\text{total}} = \sum_{i=1}^{N_s}\left(T_c + T_s^i\right). \tag{22}$$

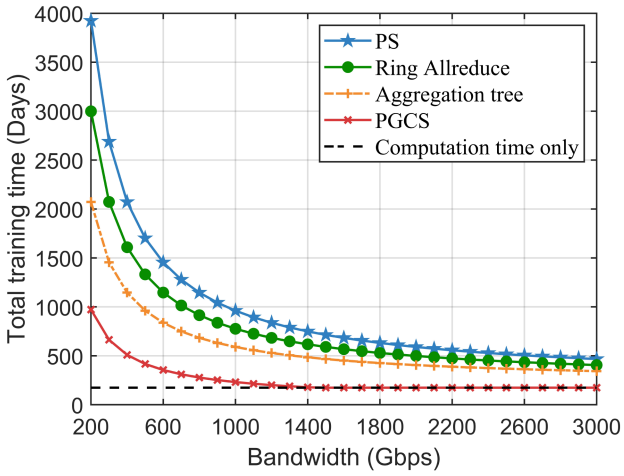


Fig. 6. Total training time vs. bandwidth.

By parallelizing the gradient computation and gradient synchronization, the total training time for our proposed schemes is given by

$$T_{\text{total}} = \sum_{i=1}^{N_s}\max\left(T_c, T_s^i\right). \tag{23}$$

Table III presents the total training time T_{total} for the proposed schemes compared to the baselines under different connections. We assume that the available bandwidth between data centers will randomly change in the range of 500 (Gbps) to 1 (Tbps). It can be seen from Table III that the proposed scheme outperforms all baselines consistently. Take the proposed scheme with arbitrary connection as an example, the total training time T_{total} of PS, Ring Allreduce, Aggregation tree, and PGCS are 3213 (days), 1049 (days), 632 (days) and 435 (days), respectively. This implies that the proposed scheme can enhance the efficiency of distributed training by 86.46%, 58.53% and 31.17%, as compared to other baselines. This is because of that, PGCS minimizes overall training time by parallelizing the gradient computation and gradient synchronization. Moreover, it can be observed from Table III that the total training time T_{total} may increase with the number of data centers. This is because of that, when the number of data centers is large, the bandwidth between data centers becomes a performance bottleneck.

To further verify the advantages of PGCS, we perform simulations under a fully connected topology, with the bandwidth between the data centers ranging from 500 (Gbps) to 3000 (Gbps). As shown in Fig. 6, when the bandwidth is limited, synchronization latency becomes the dominant component of the total training time, significantly reducing the overall training efficiency. Although PGCS also causes computing resources to be idle, it consistently maintains better performance. In addition, BLP optimizes the gradient synchronization strategy, further reducing the gradient synchronization latency. As bandwidth increases, baselines benefit primarily by reducing synchronization latency. However, the PGCS exhibits a different performance pattern. When the synchronization time becomes shorter than the

TABLE III
THE TOTAL TRAINING TIME (IN DAYS) UNDER DIFFERENT CONNECTION TOPOLOGIES

Method	Fully connection	Mesh connection	Ring connection	Arbitrary connection
PS	1253	1218	3260	3213
Ring AllReduce	995	1013	1024	1049
Aggregation Tree	737	702	679	632
PGCS	305	434	477	435

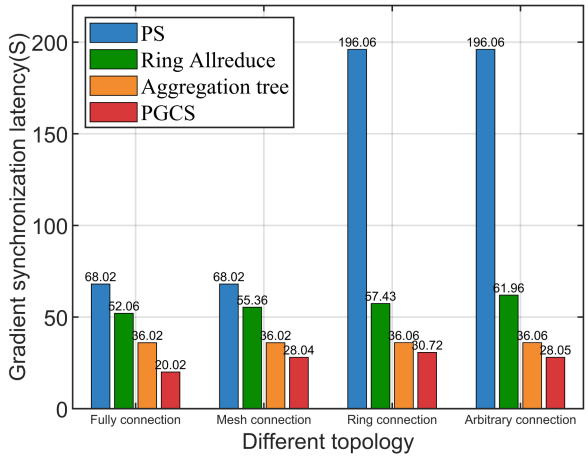


Fig. 7. Gradient synchronization latency vs. connections.

computation time, PGCS is able to completely overlap synchronization with computation. In such cases, the synchronization latency is effectively hidden. These results show that PGCS remains effective in both communication- and computation-dominated scenarios.

Gradient synchronization latency: In Fig. 7, when the available bandwidths between data centers are 1 (Tbps), the relationship between the gradient synchronization latency T_s^i and the network connections is depicted. it can be seen from Fig. 7 that PGCS consistently exhibits robust effectiveness under all connections, whereas other schemes only maintain good performance under specific connections. For example, when the topology is fully connection, the gradient synchronization latency T_s^i of the PS and PGCS are 68.02 (s) and 20.02 (s), respectively. However, when the topology connections is ring connection, the gradient synchronization latency T_s^i of the PS and PGCS are 196.06 (s) and 30.72 (s). This is due to the fact that we choose the paths by considering the dynamics of network connections and bandwidth rather than just specific scenarios.

The gradient synchronization latency T_s^i under different bandwidths is presented in Fig. 8. In particular, it can be seen that our scheme exhibits superior performance consistently in all cases. For example, when $B_l = 1500$ (Gbps) and the topology is mesh connection, the gradient synchronization latency of PS, Ring Allreduce, Aggregation tree, and PGCS are 46.69 (s), 38.29 (s), 25.35 (s) and 20.04 (s), respectively. This is because of that, we take advantage of the fact that the gradients can be partitioned and the size remains constant after reduction, and use a tree-based path to minimize the amount of transmitted gradient. Moreover, the gradient synchronization latency T_s^i of all schemes decreases as the bandwidth B_l increases, which is intuitively expected. Importantly, although the performance gap becomes smaller with increasing bandwidth, our method consistently outperforms all other compared methods.

Training Loss: The general form of the scaling law for training loss L is given by

$$L(N_m, F_{\min}) = \left(\frac{N_c}{N_m}\right)^{\alpha_N} + \left(\frac{F_c}{F_{\min}}\right)^{\alpha_F}, \tag{24}$$

Here, N_c and F_c indicate the critical model size and training step size, respectively, beyond which further increases yield diminishing returns. α_N and α_F are power-law exponents that determine the rate at which loss decreases with increasing model and training step sizes. $F_{\min}$ is the minimum number of steps necessary to reach L. In addition, $F_{\min}$ admits the following equations

$$F_{\min}(F, B_{\text{size}}) = \frac{F}{1 + B_{crit}(L_o)/B_{\text{size}}}, \tag{25}$$

where F is the number of training steps, B_{size} represents the batch size, B_{crit} indicate the critical batch size. When $L_o = 0.5$, $B_{crit} = 5.7 \times 10^9$. Substituting equation (25) into equation (24), the training loss L is given by

$$L(N_m, F) = \left(\frac{N_c}{N_m}\right)^{\alpha_N} + \left(\frac{F_c}{F}\right)^{\alpha_F} \cdot \left(1 + \frac{B_{crit}(L_o)}{B_{\text{size}}}\right)^{\alpha_F}. \tag{26}$$

According to [60], $N_c = 8.8 \times 10^{13}$, $F_c = 2.1 \times 10^3$, and α_N, α_F are 0.077, and 0.76, respectively.

When the available bandwidth between data centers are 1 (Tbps), Fig. 9 shows the relationship between the training loss and time for different connections. It is clear that distributed training schemes do not affect training loss, and ultimately all schemes achieve nearly the same training loss. Additionally, it can be seen that the training loss of PGCS decreases faster than the other three schemes in Fig. 9(a), (b), (c) and (d). For example, when the topology is arbitrary connection, the PS, Ring Allreduce, Aggregation tree, and PGCS take about 164 (days), 56 (days), 35 (days), and 23 (days), respectively, to achieve same training loss (around 10). This is because PGCS parallelizes the gradient computation and gradient synchronization and makes efficient use of the bandwidth.

4) Ablation Studies: We conducted ablation simulations to evaluate the individual contributions of the PGCS strategy and the BLP algorithm to overall performance. These simulations allow us to quantify the impact of removing these key components:

w/o BLP: This variant replaces the BLP algorithm with a standard Ring Allreduce approach for gradient synchronization.

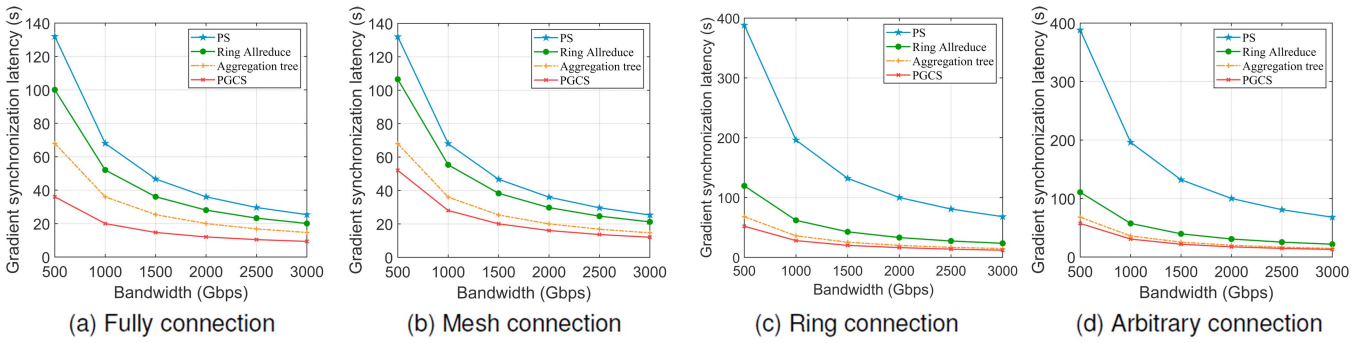


Fig. 8. Gradient synchronization latency vs. bandwidths.

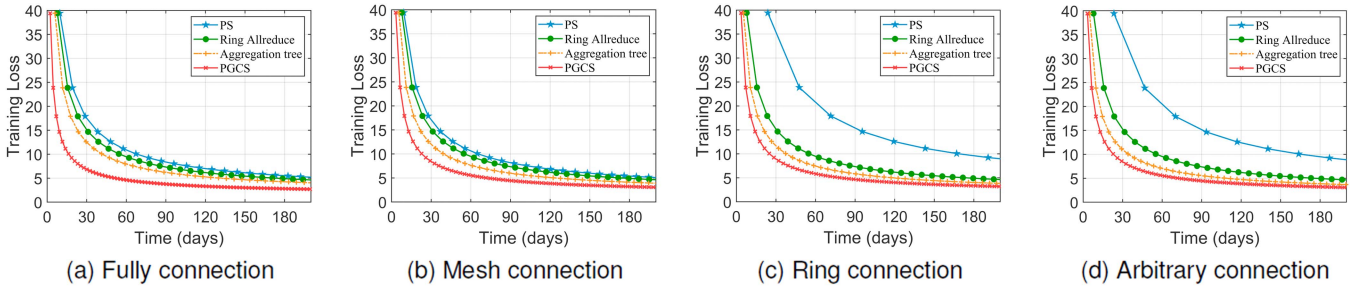


Fig. 9. Training loss vs. time.

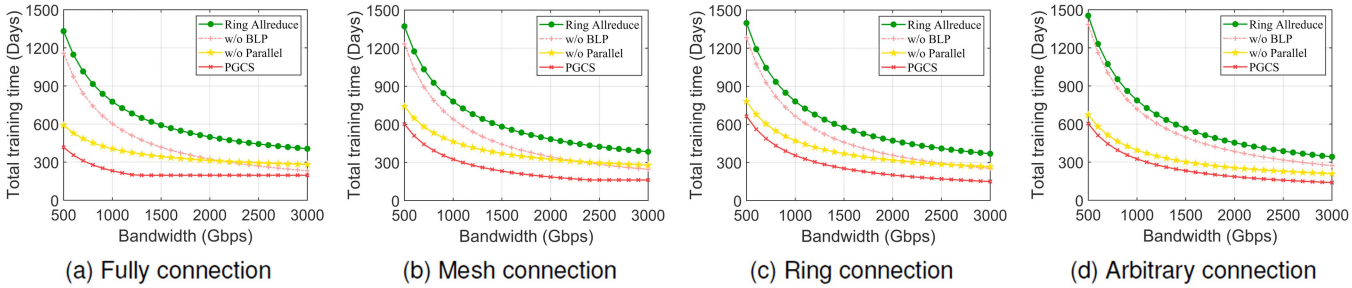


Fig. 10. The results of the PCSG and its variants in the ablation experiment.

w/o Parallel: In this variant, the parallelization of gradient computation and synchronization is disabled, reverting to a standard synchronization mechanism.

In Fig. 10, the relationship between the total training time and bandwidth for different connections is depicted. Firstly, it is obvious that both key components provide significant performance gains. For example, when $B_l = 1500$ (Gbps) and the topology is ring connection, the total training time of Ring Allreduce, w/o BLP and w/o Parallel are 575 (days), 459 (days) and 369 (days), respectively. In addition, it can be seen that the training time of w/o BLP and PGCS does not keep decreasing as the bandwidth increases. This is because of that, by parallelizing the gradient computation and gradient synchronization, the time of a training step is equal to the large value of the gradient synchronization time and the gradient computation time. The gradient synchronization time decreases as the bandwidth continues to increase, when the gradient synchronization time is less than the gradient computation time, the training time does not decrease with increasing bandwidth. Last but not least, when the bandwidth is small, the training time of w/o BLP will be longer than w/o Parallel; when the bandwidth is large, the training time for w/o BLP will be shorter than w/o Parallel. This is because the gradient synchronization time becomes the bottleneck for efficiency gains when the bandwidth is small, and the gradient computation time becomes the bottleneck when the bandwidth is large.

B. Experiments

1) Experiment Setup: This subsection describes the physical experimental environment used to evaluate the performance of PGCS and BLP.

Experimental Platform: The experimental platform consists of four identical servers, each equipped with four NVIDIA RTX 4090 GPUs. These servers are interconnected through a Wedge 100BF32X switch with a P4-programmable data plane. The role

TABLE IV
THE TOTAL TRAINING TIME (IN MINUTES) UNDER DIFFERENT BANDWIDTH CONFIGURATIONS

Method	10 Gbps	40 Gbps	100 Gbps	Mixed
PS	82.79	46.05	38.87	45.74
Ring AllReduce	72.02	44.47	39.15	72.25
Aggregation Tree	58.50	39.93	36.19	43.51
w/o PGCS	58.17	40.04	36.26	36.81
w/o BLP	33.07	32.95	33.02	39.11
PGCS	**32.92**	**32.80**	**32.84**	**32.93**

TABLE V
THE PERPLEXITY OF MERGED MODELS

Method	Science (PubMed)	Code (GitHub)
PS	10.42	6.59
Ring AllReduce	10.93	6.36
Aggregation Tree	10.48	6.92
w/o PGCS	10.34	6.89
w/o BLP	10.18	6.78
PGCS	**10.31**	**6.67**

of programmable switches is to simulate bandwidth variations between nodes and to implement the forwarding behavior of different gradient synchronization algorithms.

Datasets and Models: To evaluate the effectiveness of the proposed method, we adopt Llama-3.2-1B as the base model. Two independent instances of the model are trained on domain-specific subsets derived from The pile [61], each comprising approximately 50 million tokens. Specifically, the science model is trained on abstracts from PubMed, while the code expert is trained on source code collected from GitHub. Each model is trained for 2 epochs with a batch size of 32 and a maximum sequence length of 3,000 tokens. A learning rate of 1e-5 is used throughout the training process.

Baselines and Metrics: We compare our approach against several representative baselines, including PS, Ring AllReduce, Aggregation Tree, and two ablation variants. The metrics used for performance evaluation are total training time and perplexity.

Experimental Details: To evaluate the performance of all schemes under different network conditions, we conduct experiments using several bandwidth configurations: the bandwidth between data centers is set to 10 (Gbps), 40 (Gbps), or 100 (Gbps), respectively. To reflect the fact that bandwidth between data centers is often inconsistent in practice, we further consider a heterogeneous setting in which the bandwidth between each data center pair is randomly selected from the specified bandwidth options. This setup enables a comprehensive evaluation of the robustness and adaptability of PGCS and BLP under various communication constraints.

2) Results and Analysis: This subsection presents the experimental results on total training time and perplexity, showing that our method outperforms all baselines.

Total training time: The total training time under different bandwidth configurations is presented in Table IV, and the experimental results are consistent with the simulation analysis. Specifically, w/o PGCS outperforms the PS, Ring AllReduce, and Aggregation Tree baselines. This is because BLP can optimize gradient synchronization paths, thereby improving bandwidth utilization between data centers and reducing synchronization latency. Moreover, it can be observed that the total training time of w/o BLP remains unchanged when the bandwidth increases from 10 (Gbps) to 100 (Gbps). The reason lies in the parallelization of gradient computation and synchronization in our method. When the bandwidth between data centers is low, the total training time is dominated by the gradient synchronization latency. As bandwidth increases, the gradient synchronization latency decreases. Once the gradient synchronization latency is shorter than the gradient computation time, the total training time is then determined by the computation time. In this case, further increases in bandwidth no longer reduce the overall training time. Finally, it should be noted that our method consistently achieves the best performance in all cases, including in heterogeneous network configurations. This is because that we explicitly account for variations in bandwidth across different data center pairs during the design stage of the BLP.

Perplexity: After completing the training of the expert models using all training methods, we employ Arcee Fusion [30] to merge the two domain-specific models. Since the merged model has not undergone supervised fine-tuning, we use perplexity as the evaluation metric. The perplexity under different training methods is presented in Table V. It can be observed that the merged models produced by all methods achieve similar perplexity. In theory, under the same data order, model, optimizer state, random seeds and hardware environment, all algorithms should converge to identical models and produce the same perplexity. However, in practice, small numerical discrepancies can arise due to the order of gradient aggregation across multiple GPUs. These discrepancies may be accumulated and amplified by stateful optimizers such as AdamW over many iterations, which is a possible cause of the slight difference in perplexity observed in our experiments. It should be noted that the LLM training process is extremely complex. In addition to the above factors, non-deterministic algorithms, parallel computation kernels, and hardware scheduling may also contribute to these discrepancies. Importantly, our method achieved the least training time for the expert model while preserving model quality and demonstrating superior training efficiency.

V. CONCLUSION

In this work, a novel training scheme named PGCS is proposed for LLMs in distributed training across multiple data centers. This scheme addresses the challenge of gradient synchronization latency by parallelizing gradient computation and synchronization. It also incorporates a gradient synchronization algorithm named BLP, which optimizes the synchronization strategy using a bisection method with linear programming. The results of simulations and experiments show that compared to traditional synchronization methods like PS, Ring Allreduce, and aggregation trees, PGCS can significantly reduce both total training time and gradient synchronization latency.

Developing more advanced parallelization strategies for dynamic network environments, incorporating critical factors

such as energy consumption, memory consumption and system stability into the design of PGCS, and integrating advanced compression techniques to reduce communication overhead without compromising model accuracy are all worthwhile future work.

REFERENCES

[1] M. A. K. Raiaan et al., "A review on large language models: Architectures, applications, taxonomies, open issues and challenges," *IEEE Access*, vol. 12, pp. 26839–26874, 2024.

[2] J. Sevilla, L. Heim, A. Ho, T. Besiroglu, M. Hobbhahn, and P. Villalobos, "Compute trends across three eras of machine learning," in *Proc. Int. Joint Conf. Neural Netw.*, Jul. 2022, pp. 1–8.

[3] H. Xiong, Y. Zhuge, J. Zhu, L. Zhang, and H. Lu, "3UR-LLM: An end-to-end multimodal large language model for 3D scene understanding," *IEEE Trans. Multimedia*, vol. 27, pp. 2899–2911, 2025.

[4] Z. Huang, S. Zhong, P. Zhou, S. Gao, M. Zitnik, and L. Lin, "A causality-aware paradigm for evaluating creativity of multimodal large language models," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 5, pp. 3830–3846, May 2025.

[5] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 3580–3599, Jul. 2024.

[6] P. Faraboschi, E. Giles, J. Hotard, K. Owczarek, and A. Wheeler, "Reducing the barriers to entry for foundation model training," 2024, *arXiv:2404.08811*.

[7] Z. Zhang et al., "CPM-2: Large-scale cost-effective pre-trained language models," *AI Open*, vol. 2, pp. 216–224, 2021.

[8] H. Mi et al., "Collaborative deep learning across multiple data centers," *Sci. China Inf. Sci.*, vol. 63, pp. 1–11, 2020.

[9] W. Dai, J. Fan, Y. Miao, and K. Hwang, "Deep learning model compression with rank reduction in tensor decomposition," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 1, pp. 1315–1328, Jan. 2025.

[10] G. Team et al., "Gemma: Open models based on gemini research and technology," 2024, *arXiv:2403.08295*.

[11] G. S. Amorim, J. Belman-Flores, R. de Paoli Mendes, O. R. Sandoval, A. Khosravi, and J. J. Garcia-Pabon, "Recent advancements in thermal management technologies for cooling of data centers," *J. Braz. Soc. Mech. Sci. Eng.*, vol. 46, no. 8, pp. 1–20, 2024.

[12] J. Zheng, G. Tao, S. Qin, D. Wang, and Z. Ma, "Intent-based multi-cloud storage management powered by a fine-tuned large language model," *IEEE Access*, vol. 13, pp. 72736–72753, 2025.

[13] F. Li, "Cloud-native database systems at alibaba: Opportunities and challenges," in *Proc. VLDB Endowment*, 2019, vol. 12, no. 12, pp. 2263–2272.

[14] M. Copeland et al., "Microsoft azure and cloud computing," in *Microsoft Azure: Planning, Deploying, and Managing the Cloud*. Berkeley, CA,USA: Springer, 2015, pp. 3–20.

[15] M. Kahng et al., "LLM comparator: Interactive analysis of side-by-side evaluation of large language models," *IEEE Trans. Vis. Comput. Graph.*, vol. 31, no. 1, pp. 503–513, Jan. 2025.

[16] X. Xie et al., "AZ-code: An efficient availability zone level erasure code to provide high fault tolerance in cloud storage systems," in *Proc. 35th Symp. Mass Storage Syst. Technol.*, 2019, pp. 230–243.

[17] L. Liu, L. Song, X. Chen, H. Yu, and G. Sun, "Accelerating model synchronization for distributed machine learning in an optical wide area network," *J. Opt. Commun. Netw.*, vol. 14, no. 10, pp. 852–865, 2022.

[18] L. Liu, H. Yu, G. Sun, L. Luo, Q. Jin, and S. Luo, "Job scheduling for distributed machine learning in optical wan," *Future Gener. Comput. Syst.*, vol. 112, pp. 549–560, 2020.

[19] K. Hsieh et al., "Gaia:{Geo-Distributed } machine learning approaching { LAN} speeds," in *Proc. 14th USENIX Symp. Networked Syst. Des. Implementation*, 2017, pp. 629–647.

[20] H. Zhou et al., "Large language model (LLM) for telecommunications: A comprehensive survey on principles, key techniques, and opportunities," *IEEE Commun. Surveys. Tuts.*, vol. 27, no. 3, pp. 1955–2005, Jun. 2025.

[21] S. Dash et al., "Optimizing distributed training on frontier for large language models," in *Proc. ISC High Perform. Res. Paper Proc. (39th Int. Conf.*, 2024, pp. 1–11.

[22] Z. Zhang and C. Wang, "MIPD: An adaptive gradient sparsification framework for distributed DNNs training," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 3053–3066, Nov. 2022.

[23] H. Li et al., "Gradient-mask tuning elevates the upper limits of LLM performance," 2024, *arXiv:2406.15330*.

[24] V. A. Korthikanti et al., "Reducing activation recomputation in large transformer models," in *Proc. Mach. Learn. Syst.*, 2023, vol. 5, pp. 341–353.

[25] M. Cho, U. Finkler, D. Kung, and H. Hunter, "Blueconnect: Decomposing all-reduce for deep learning on heterogeneous network hierarchy," in *Proc. Mach. Learn. Syst.*, 2019, vol. 1, pp. 241–251.

[26] F. Dai, Y. Chen, Z. Huang, and H. Zhang, "WRHT: Efficient all-reduce for distributed DNN training in optical interconnect systems," in *Proc. 52nd Int. Conf. Parallel Process.*, 2023, pp. 556–565.

[27] M. Li et al., "Branch-train-merge: Embarrassingly parallel training of expert language models," 2022, *arXiv:2208.03306*.

[28] S. Sukhbaatar et al., "Branch-train-mix: Mixing expert LLMs into a mixture-of-experts LLM," 2024, *arXiv:2403.07816*.

[29] C. Liu et al., "An adaptive and scalable framework for resource-efficient deployment of mixture of experts in LLM-based intelligent IoT networks," *IEEE Internet Things J.*, vol. 12, no. 13, pp. 22746–22757, Jul. 2025.

[30] C. Goddard et al., "Arcee's mergekit: A toolkit for merging large language models," in *Proc. Conf. Empirical Methods Natural Lang. Process.: Ind. Track*, 2024, pp. 477–485.

[31] A. Nabli, L. Fournier, P. Erbacher, L. Serrano, E. Belilovsky, and E. Oyallon, "ACCO: Accumulate while you communicate, hiding communications in distributed LLM training," 2024, *arXiv:2406.02613*.

[32] L. Sun et al., "TinyR1-32B-preview: Boosting accuracy with branch-merge distillation," 2025, *arXiv:2503.04872*.

[33] R. Bansal et al., "LLM augmented llms: Expanding capabilities through composition," 2024, *arXiv:2401.02412*.

[34] F. Wan, X. Huang, D. Cai, X. Quan, W. Bi, and S. Shi, "Knowledge fusion of large language models," 2024, *arXiv:2401.10491*.

[35] R. Li, P. Shi, H. Wang, and S. Cheng, "Jointflow: Optimizing service deployment for large-scale model workflows in jointcloud," in *Proc. IEEE Int. Conf. Web Serv.*, 2024, pp. 1397–1399.

[36] B. Wang, Q. Xu, Z. Bian, and Y. You, "Tesseract: Parallelize the tensor parallelism efficiently," in *Proc. 51st Int. Conf. Parallel Process.*, 2022, pp. 1–11.

[37] Y. Huang et al., "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, vol. 32, pp. 1–11.

[38] Z. Zhang et al., "MPMoe: Memory efficient MoE for pre-trained models with adaptive pipeline parallelism," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 6, pp. 998–1011, Jun. 2024.

[39] Z. Zhang et al., "FedCSpc: A cross-silo federated learning system with error-bounded lossy parameter compression," *IEEE Trans. Parallel Distrib. Syst.*, vol. 36, no. 7, pp. 1372–1386, Jul. 2025.

[40] J. Lee, I. Hwang, S. Shah, and M. Cho, "Flexreduce: Flexible all-reduce for distributed deep learning on asymmetric network topology," in *Proc. 57th ACM/IEEE Des. Automat. Conf.*, 2020, pp. 1–6.

[41] L. Chen, Z. Xiang, K. Lei, and X.-Y. Zhang, "Multi-task model fusion via adaptive merging," in *Proc. ICASSP IEEE Int. Conf. Acoust., Speech Signal Process.*, 2025, pp. 1–5.

[42] J. Frankle, G. K. Dziugaite, D. Roy, and M. Carbin, "Linear mode connectivity and the lottery ticket hypothesis," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 3259–3269.

[43] J. Zhang et al., "Composing parameter-efficient modules with arithmetic operation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, vol. 36, pp. 12589–12610.

[44] M. S. Matena and C. A. Raffel, "Merging models with fisher-weighted averaging," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 17703–17716.

[45] F. Zeng, W. Gan, Y. Wang, and P. S. Yu, "Distributed training of large language models," in *Proc. IEEE 29th Int. Conf. Parallel Distrib. Syst.*, 2023, pp. 840–847.

[46] S. Dash et al., "Optimizing distributed training on frontier for large language models," in *Proc. ISC High Perform. Res. Paper Proc. 39th Int. Conf.*, 2024, pp. 1–11.

[47] P. Barham et al., "Pathways: Asynchronous distributed dataflow for ML," in *Proc. Mach. Learn. Syst.*, 2022, vol. 4, pp. 430–449.

[48] Q. Xu and Y. You, "An efficient 2D method for training super-large deep learning models," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, 2023, pp. 222–232.

[49] F. Strati, P. Elvinger, T. Kerimoglu, and A. Klimovic, "Ml training with cloud GPU shortages: Is cross-region the answer?," in *Proc. 4th Workshop Mach. Learn. Syst.*, 2024, pp. 107–116.

[50] J. Chen, J. Wang, S. Guo, J. Hao, X. Qiu, and Z. Xiong, "A transformer-block-wise collaborative training mechanism with hybrid parallelism over heterogeneous networks," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2025, pp. 01–06.
[51] J. Duan et al., "Efficient training of large language models on distributed infrastructures: A survey," 2024, *arXiv:2407.20018*.
[52] T. Zhang, H. Yu, Z. Yang, Y. Chen, and S. Yu, "LaVFL: Efficient verifiable federated learning for large language models," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 6, pp. 6214–6229, Nov.-Dec. 2025.
[53] H. Jin, "Communication-efficient distributed LLM training with adaptive traffic control," in *Proc. IEEE Int. Conf. Big Data*, 2024, pp. 8685–8687.
[54] Z. Zhang, C. Wu, and Z. Li, "Near-optimal topology-adaptive parameter synchronization in distributed DNN training," in *Proc. IEEE INFOCOM 2021-IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
[55] D. Yu et al., "MoESys: A distributed and efficient mixture-of-experts training and inference system for internet services," *IEEE Trans. Serv. Comput.*, vol. 17, no. 5, pp. 2626–2639, Sep./Oct. 2024.
[56] H. Che, H. Jin, Z. Gu, Y. Lin, C. Jin, and H. Chen, "LLM-driven medical report generation via communication-efficient heterogeneous federated learning," *IEEE Trans. Med. Imag.*, early access, Jul. 21, 2025, doi: 10.1109/TMI.2025.3591185.
[57] B. Hanindhito, B. Patel, and L. K. John, "Bandwidth characterization of deepspeed on distributed large language model training," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2024, pp. 241–256.
[58] F. Dai, Y. Chen, Z. Huang, H. Zhang, and F. Zhang, "Efficient all-reduce for distributed DNN training in optical interconnect systems," in *Proc. 28th ACM SIGPLAN Annu. Symp. Princ. Pract. Parallel Program.*, 2023, pp. 422–424.
[59] W. Wang, M. Ghobadi, K. Shakeri, Y. Zhang, and N. Hasani, "Optimized network architectures for large language model training with billions of parameters," 2023, *arXiv:2307.12169*.
[60] J. Kaplan et al., "Scaling laws for neural language models," 2020, *arXiv:2001.08361*.
[61] L. Gao et al., "The pile: An 800gb dataset of diverse text for language modeling," 2020, *arXiv:2101.00027*.



Hao Li received the B.Eng. degree in electronic information engineering from the Wuhan University of Science and Technology, Wuhan, China, in 2021, and the M.Eng. degree from the Electronic Information School, Wuhan University, Wuhan, China, in 2023. He is currently working toward the Ph.D. degree with the Electronic Information School, Wuhan University. His current research interests include artificial intelligence, distributed network simulation and software designed networks.



Hao Jiang (Member, IEEE) received the B.Eng. degree in communication engineering and the M.Eng. and Ph.D. degrees in communication and information systems from Wuhan University, Wuhan, China, in 1999, 2001, and 2004, respectively. From 2004 to 2005, he was a Postdoctoral Researcher with LIMOS, Clermont-Ferrand, France. He was a Visiting Professor with the University of Calgary, Calgary, AB, Canada, ISIMA, and Blaise Pascal University, Clermont-Ferrand. He is currently a Professor with Wuhan University. He has authored or coauthored more than 60 papers in different journals and conferences. His research interests include mobile ad hoc networks and mobile.



Jing Wu (Member, IEEE) received the B.Eng. degree in communication engineering and the Ph.D. degree in communication and information systems from Wuhan University, Wuhan, China, in 2002 and 2007, respectively. From 2004 to 2005, she was a Postdoctoral Researcher with LIMOS, ClermontFerrand, France. She is currently an Associate Professor with Wuhan University. Her research interests include wireless communication networks, network simulation, and intelligence data processing.



Guiao Yang received the B.Eng. degree in electronic information engineering from the Wuhan University of Technology, Wuhan, China, in 2023. He is currently working toward the master's degree with Wuhan University, Wuhan, China. His research interests include network simulation and intelligent networking systems. During his studies, he was the recipient of the multiple honors, including National Endeavor Scholarship and the university's "Outstanding Student" award, and won first prizes in both the National University Mathematical Modeling Competition and the China University Intelligent Robotics Competition.



Jian Zhang (Member, IEEE) received the B.Eng. degree from Southeast University, Nanjing, China, in 1999, and the Ph.D. degree from the Shanghai Institute of Microsystem and Information Technology, Chinese Academy of Sciences (CAS), Shanghai, China, in 2008. From 2010 to 2012, he was a Marie Curie Experienced Researcher with the Institute of Electronics and Communications and Information Technology (ECIT), Queens University of Belfast, Belfast, U.K., where he was involved in the development of SiGe MMIC. Currently, He is currently a Full Professor with Wuhan University, His main research focus on wireless communication productions development using millimeter-wave and Tera-Hz technology.